HASH1: Hashtabel search hits en misses

Manoah Kinnaert

April 27, 2026

1 Korte inleiding

In dit verslag trachten we beide formules voor het gemiddeld aantal search hits en misses van hashtabellen die gebruik maken van lineaire probing experimenteel te valideren. We proberen ook te evalueren of deze formules een goede inschatting geven van het gemiddelde aantal search hits en search misses. Alle code die gebruikt is voor dit experiment is hier te vinden: https://github.com/ManoahKinnaert/HASH.

2 Verloop van het experiment

Voor het uitvoeren van het experiment is er gekozen om te werken met hashtabellen met grootte 100000. We voeren meerdere individuele metingen uit waarbij we telkens een nieuwe lege tabel initializeren en deze vullen tot een bepaalde load factor bereikt is. In ons geval doen we dit voor een load factor van 0.10 tot en met een load factor van 0.90 waarbij we telkens (voor iedere nieuwe meting) de load factor laten stijgen met 0.05. Nadat deze hashtabel dan gevuld is meten we hits door het aantal hitprobes te tellen. We doen dit 20000 keer per load factor en nemen dan het gemiddelde immers de te valideren formules over gemiddelden gaan. Het aantal misses wordt ook 20000 keer per load factor bepaald en hier wordt dan ook een gemiddelde voor genomen. Na de meting(en) plotten we deze datapunten en vergelijken we ze met de theoretische curves om dan onze conclusies te trekken. Aangezien de implementatie in Java gemaakt is en ik vooral bekend ben met de matplotlib module in python, exporteer ik eerst de meetpunten naar een CSV bestand. Vanuit Java wordt er dan een python scriptje gestart dat dit CSV bestand inleest en de gegevens zal plotten.

3 Data plotten

Voor het weergeven van de data in de grafieken is er gekozen voor lineair geschaalde assen. Er werd geen voordeel verwacht voor het gebruikt van bv een log log plot. Lineair geschaalde assen zouden perfect moeten volstaan om de nauwkeurigheid van beide formules te valideren en eventuele afwijkingen die zich zouden voordoen als de load factor groter en groter wordt waar te nemen.

4 Verwachtingen

Zoals eigenlijk eerder al aangehaald is verwachten we wel dat de theoretische curves zouden moeten samenvallen met onze meetpunten (of toch dicht in de buurt liggen). Voor het gemiddelde aantal search hits verwachten we $\frac{1}{2}\frac{1}{1-\alpha}$ hits. En voor het gemiddelde aantal search misses verwachten we $\frac{1}{2}\frac{1}{(1-\alpha)^2}$. Maar we verwachten dit enkel maar tot een load factor van 0.5. Eens die grens van 0.5 voorbij zal de nauwkeurigheid van deze formules verminderen.

5 Bevindingen

Voor het gemiddelde aantal search hits zien we dat de theoretische curve tot en met een load factor van 0.5 lijkt samen te vallen met onze bevindingen. Naarmate de load factor groter wordt zien we echter wel wat duidelijkere afwijkingen (hoewel deze minimaal lijken te zijn). Zoals 1 laat zien, merken we een exponentiële groei in het aantal search hits en search misses naarmate de load factor groter wordt. Dit is uiteraard gedrag dat we willen vermijden, maar hoe krijgen we dit voor elkaar?

6 Wat als...

Bij ons experiment kozen we er bewust voor om de array niet te rehashen zodat we het effect van de load factor konden waarnemen op het aantal search hits en misses. We zagen dan duidelijk een drastische stijging in het aantal search hits en search misses wanneer de hashtable steeds voller en voller werd. De boodschap is duidelijk, we willen het aantal search hits en search misses zo laag mogelijk houden. Dit kan bereikt worden door vanop een bepaalde load factor (in ons geval 0.5) de hashtabel groter te maken en te rehashen. Dat wil zeggen we maken een nieuwe hashtabel aan die tweemaal zo groot is. We nemen de sleutels uit de oude hashtabel en zetten die in onze nieuwe nieuwe hashtabel, deze gaan dan uiteraard opnieuw door een modulaire hashfunctie om zo in onze nieuwe hashtabel te geraken. Zoals 2 illustreert zien we dan dat het aantal search hits en search misses vrij klein blijft wat de performantie verbetert. Natuurlijk kost het rehashen op zich ook wel iets maar het is zeker die kost wel waard als het aantal search hits en misses in het andere geval exponentieel zou groeien. Merk op dat de punten na het rehashen bewust verkeerd zijn aangebracht op de grafiek om te illustreren dat we wel

gerehashed hebben op het punt waar onze eerste hashtabel niet gerehashed zou zijn (en dus een hogere load factor zou hebben).

7 Een goede benadering?

De formules $\frac{1}{2}\frac{1}{1-\alpha}$ voor het gemiddelde aantal search hits en $\frac{1}{2}\frac{1}{(1-\alpha)^2}$ voor het gemiddelde aantal search misses zijn zeker goede benaderingen. Toch als we de kanttekening maken dat de loadfactor kleiner of gelijk aan 0.5 blijft. En eerlijk gezegd zelfs als de loadfactor groter dan 0.5 wordt lijken de resultaten bekomen uit de formule en het experiment niet zo ver uit elkaar te liggen. We meten de fout hier door de absolute waarde te nemen van het verschil tussen de theoretische probes en de gemeten probes. Opmerkelijk is dat deze fout stijgt zoals 3 illustreert, en al zeker wanneer de load factor groter is dan 0.5. Onder een load factor van 0.5 lijkt de fout niet zo sterk te evolueren. Het gebruik van de formules voor een load factor onder 0.5 ziet er dus uit als een goede benadering voor het gemiddelde aantal search hits en misses.

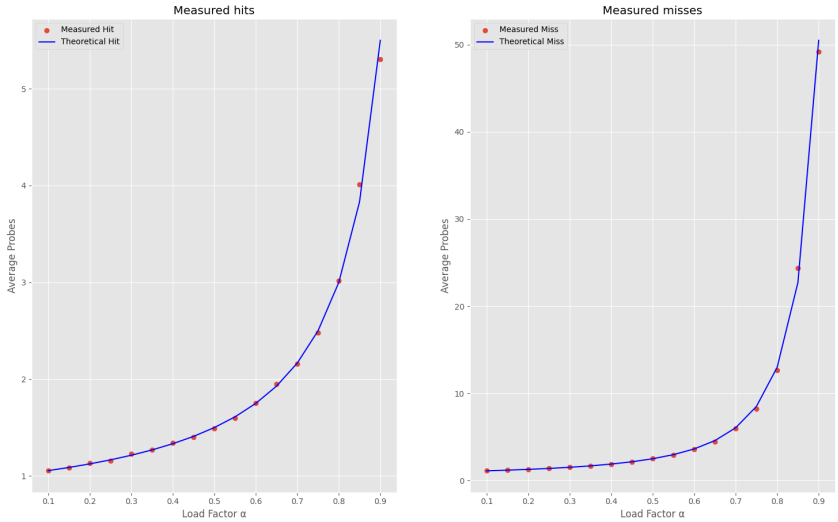

Figure 1: Hashtabel searh hits en search misses zonder resize bij loadfactor groter dan 0.5 (indien niet duidelijk genoeg zie git repo)

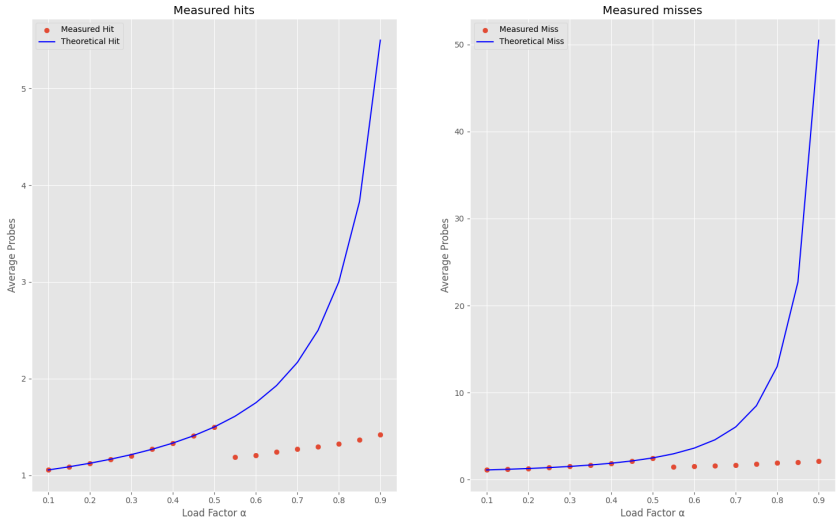
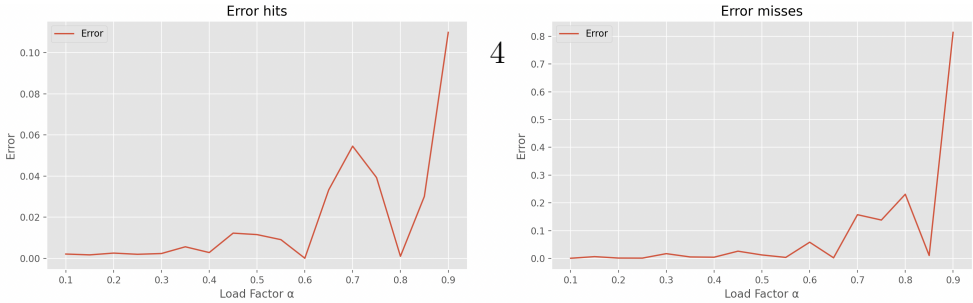
Figure 2: Hashtabel searh hits en search misses met resize bij loadfactor groter dan 0.5, de x-as is bewust foutief (indien niet duidelijk genoeg zie git repo)